

LECTURE 1 • TUESDAY 8 SEPTEMBER 2026

Multilayer Perceptrons and Optimisation

neural networks, and the problem of finding them

Carolina Cuesta-Lazaro • PHYS-GA 2043

Transcribed and typeset by Claude from the lecture board notes

1. This week in AI4Science

Links shown in class

- T. Buckmaster, statement of 8 September 2026 (Courant Institute) — <https://cims.nyu.edu/~tristanb/statement.pdf>
- OpenAI, *On the Navier–Stokes Millennium Prize Problem* — <https://openai.com/index/navier-stokes-solution/>
- *Quanta Magazine*, 8 September 2026 — <https://www.quantamagazine.org/ai-has-solved-one-of-maths-1-million-millennium-prize-problems-20260908/>

Both announcements came out on the morning of this lecture.

Buckmaster (NYU Courant) and Alpöge (Anthropic) published proofs of finite-time blowup with smooth forcing for incompressible porous media, Boussinesq, and 3D incompressible Euler. They used AI models throughout and verified the results in Lean. Twelve hours later OpenAI announced a proof of a finite-time singularity for forced 3D Navier–Stokes, also formalised in Lean, obtained by about 10^4 agents running for 88 hours, plus 17 hours to formalise. The two groups then disputed priority; the links cover both sides.

Lean is a proof assistant. It reduces every step of an argument to a small fixed set of axioms and checks those steps mechanically, so we can know a proof is correct even when it is far too long for anyone to read.

2. The supervised learning problem

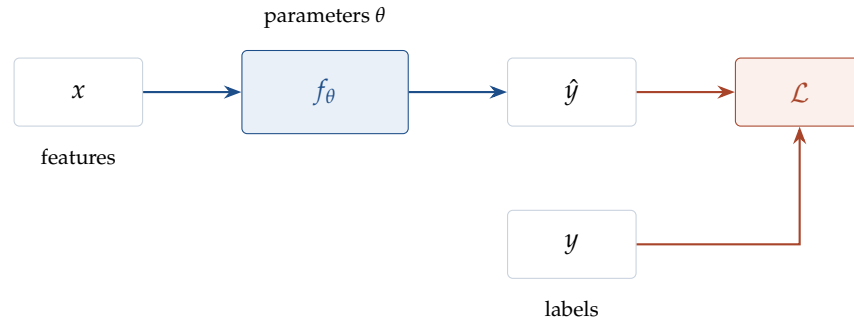
We are given pairs of **features** and **labels**.

$$x \in \mathbb{R}^m \quad (\text{features}), \quad y \in \mathbb{R}^n \quad (\text{labels}).$$

We want a function that maps one to the other. We do not know it, so we choose a family of candidate functions indexed by parameters θ and search inside that family:

$$\hat{y} = f_{\theta}(x), \quad f_{\theta} : \mathbb{R}^m \rightarrow \mathbb{R}^n.$$

A **loss** $\mathcal{L}(\theta; y, \hat{y})$ measures how good a given θ is. Training means minimising it.



This leaves two questions. What should f_θ look like (Sections 3–5), and how do we search for θ (Sections 7–15)?

m	input dimension	D	number of parameters
n	output dimension	g_t	gradient at step t
L	number of layers (depth)	η	learning rate / step size
θ	all parameters, $\theta \in \mathbb{R}^D$	B	minibatch size
$\mathcal{L}$	loss	ε	small constant, $\sim 10^{-8}$

3. The simplest choice, and why it fails

Start with the smallest family that maps $\mathbb{R}^m$ to $\mathbb{R}^n$: an affine map.¹

$$f_\theta(x) = Wx + b, \quad W \in \mathbb{R}^{n \times m}, \quad b \in \mathbb{R}^n, \quad \theta = \{W, b\}.$$

This can be solved in closed form, but it can only represent a linear function. The obvious way to make it richer is to stack several of them. That does not work. Compose two layers:

$$f_2(f_1(x)) = W_2(W_1x + b_1) + b_2 = \underbrace{(W_2W_1)}_{\tilde{W}}x + \underbrace{(W_2b_1 + b_2)}_{\tilde{b}},$$

which is again of the form $\tilde{W}x + \tilde{b}$. Induction extends this to any depth:

$$f_L(f_{L-1}(\cdots f_1(x) \cdots)) = W_{\text{eff}}x + b_{\text{eff}}, \quad W_{\text{eff}} = W_L W_{L-1} \cdots W_1, \quad \theta = \{\theta_1, \dots, \theta_L\}.$$

A composition of affine maps is an affine map. An L -layer linear network represents the same set of functions as a single linear layer, using L times as many parameters. Depth on its own adds nothing.

If an intermediate layer is narrow, the rank of W_{eff} is limited by it, and the stack represents less than a single layer would. We therefore need something between the layers to stop them collapsing.

4. The perceptron

We insert a nonlinearity, a scalar function A applied elementwise:

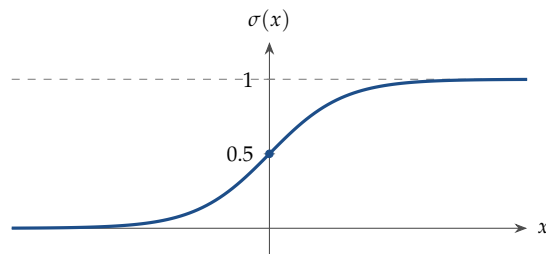
$$f_\theta(x) = A(Wx + b), \quad \theta = \{W, b\}.$$

A is the **activation**. Two common choices:

¹Board note: On the board W was labelled $m \times n$; here it is written $n \times m$ so that Wx has the shape of y .

Sigmoid

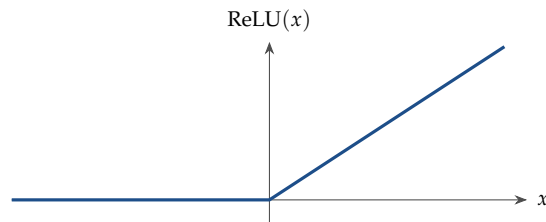
$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



The sigmoid is smooth and monotone and maps $\mathbb{R}$ to $(0, 1)$, with $\sigma(0) = \frac{1}{2}$. Its derivative $\sigma' = \sigma(1 - \sigma)$ is at most $\frac{1}{4}$ and goes to zero in both tails, so the sigmoid saturates. This matters later: when we differentiate through L layers next lecture, this factor appears L times.

ReLU

$$\text{ReLU}(x) = \begin{cases} x, & x > 0 \\ 0, & x \leq 0 \end{cases}$$



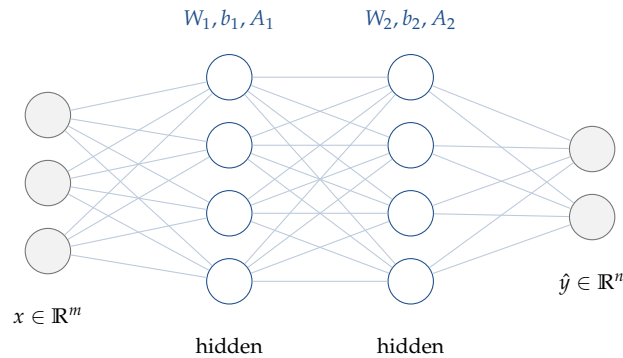
ReLU is piecewise linear, its derivative is 0 or 1, it does not saturate for $x > 0$, and it is cheap to evaluate. It is not differentiable at the origin, which does not cause problems in practice. Any non-polynomial A prevents the collapse of Section 3.

5. The multilayer perceptron

With a nonlinearity in place, stacking layers does add expressive power. Writing $f_{\theta_\ell}(u) = A_\ell(W_\ell u + b_\ell)$ for one layer:

$$f_\theta(x) = f_{\theta_L}(f_{\theta_{L-1}}(\cdots f_{\theta_1}(x) \cdots)), \quad \theta = \{\theta_1, \dots, \theta_L\}.$$

L is the **depth** and the number of rows of W_ℓ is the **width** of layer ℓ . Layers between the input and the output are called hidden. The last layer usually has no activation for regression, or a softmax for classification.



6. The universal approximation theorem

Is this family rich enough? Yes.

Universal approximation

A feedforward network with a *single hidden layer* and a suitable non-polynomial activation can approximate any continuous function on a compact subset of $\mathbb{R}^m$ to any desired accuracy, given enough hidden units.

It is easy to read more into this than it says. The question put to the class was:

What doesn't it say?

- 1. It does not say the solution can be found.** The theorem says a good θ exists somewhere in $\mathbb{R}^D$. Whether a procedure starting from a random initialisation reaches it is a separate question.
- 2. It does not say the solution will generalise.** The approximation is guaranteed on the region where we match the function, and says nothing about points outside it.

It also says nothing about cost, since “enough hidden units” can mean exponentially many, and it gives no reason to prefer depth, which is what works in practice. What makes the problem tractable is choosing an architecture whose structure matches the structure of the data. These are the inductive biases, and we take them up from Lecture 3 onwards.

Those two gaps are the syllabus:

Gap	Question	Where
Can it be found?	optimisation — find a good θ as fast as possible	rest of this lecture
Will it generalise?	regularisation, capacity, double descent	Lecture 2 onwards

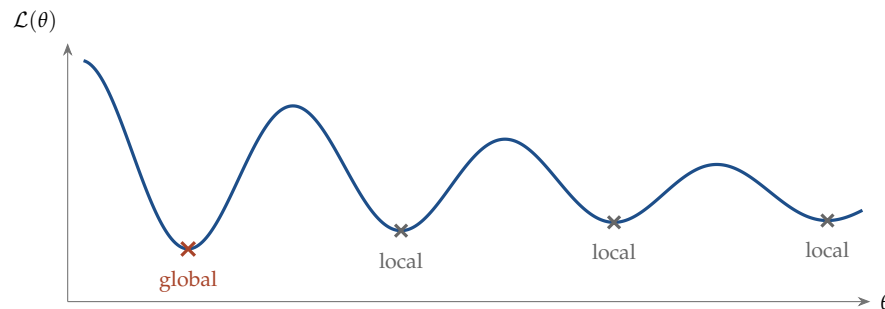
Today we deal with the first one.

7. Can it be found?

Formally, we want

$$\theta^* \in \arg \min_{\theta \in \Theta \subseteq \mathbb{R}^D} \mathcal{L}(\theta),$$

where D , the number of parameters, is typically in the millions or billions. The loss surface is not convex:



If θ^* is a local minimum, then

$$\nabla \mathcal{L}(\theta^*) = 0, \quad \text{and} \quad \nabla^2 \mathcal{L}(\theta^*) \succeq 0, \quad H_{ij} = \frac{\partial^2 \mathcal{L}}{\partial \theta_i \partial \theta_j},$$

with H positive semi-definite at that point. Both conditions are correct, but neither helps us here:

- $\nabla \mathcal{L}(\theta) = 0$ is a system of D coupled nonlinear equations with no closed-form solution.
- H has D^2 entries. At $D = 10^9$ that is 10^{18} numbers, so we cannot form it, store it or invert it. Second-order methods are not available at this scale.

Instead of solving for θ^* , we iterate, using only first-order information:

$$\theta_{t+1} = \theta_t + \eta d_t$$

d_t : descent direction η : learning rate, or step size

Each algorithm below is a different choice of d_t and of how to scale it.

8. A wish list

Before looking at specific algorithms, here is what we want from one.

Must have	Nice to have
1. Converges fast. Few steps, and each step cheap. Both matter: halving the number of steps is no good if each one costs ten times as much. 2. Lightweight in memory. The state it carries should be a small constant number of copies of θ . Anything that scales like D^2 is unusable.	1. Theoretical guarantees. Ideally a proof of convergence to a local minimum, with a rate. 2. Robust to the starting point. Performance should not depend strongly on the initialisation. 3. Insensitive to hyperparameters. It should work at its default settings on a problem nobody has tuned it for.

The last item may look like a convenience, but it is most of the reason Adam became the default optimiser. It is rarely the best choice for a given problem, and rarely a bad one for an unfamiliar problem.

9. A1 — Gradient descent

A1 Gradient descent

$$d_t = -\nabla_{\theta} \mathcal{L}(\theta) \Big|_{\theta_t} \implies \mathcal{L}(\theta + \eta d_t) < \mathcal{L}(\theta)$$

for η small enough.

A first-order Taylor expansion gives $\mathcal{L}(\theta + \eta d) \approx \mathcal{L}(\theta) + \eta \nabla \mathcal{L} \cdot d$. Choosing $d = -\nabla \mathcal{L}$ makes the correction $-\eta \|\nabla \mathcal{L}\|^2 < 0$, so the loss decreases as long as η is small enough for the expansion to hold. Of all directions of a fixed length, this one decreases the loss fastest locally. The word “locally” is where the difficulty comes from.

What goes wrong: conditioning

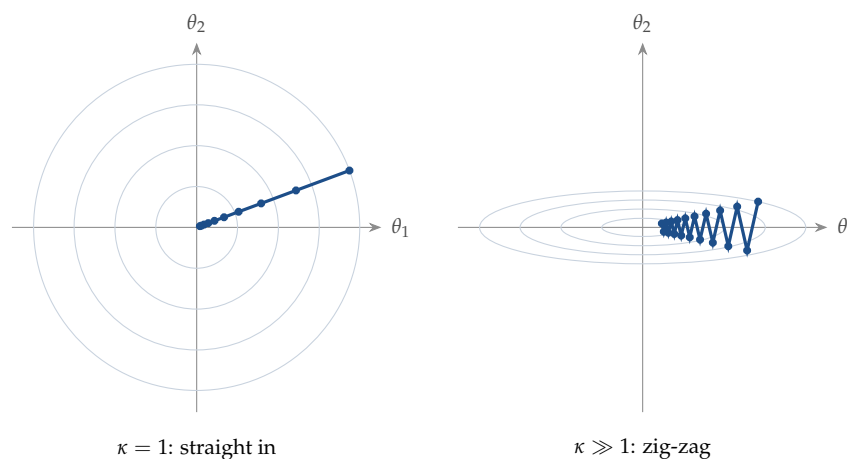
Take a quadratic with two different curvatures,

$$f(\theta) = \frac{1}{2}(\theta_1^2 + \kappa \theta_2^2), \quad \kappa > 1.$$

From a point θ , the direction pointing straight at the minimum is $-(\theta_1, \theta_2)$. Gradient descent instead moves along

$$-\nabla f = -(\theta_1, \kappa \theta_2).$$

The two agree only when $\kappa = 1$. When $\kappa \gg 1$ the steep direction dominates the gradient, which then points across the valley instead of along it:



The two directions place opposite demands on the step size. Stability along the steep direction requires $\eta < 2/\kappa$, while progress along the shallow direction goes like $(1 - \eta)^t$ and so wants η as large as possible. The steep direction therefore sets the step size and the shallow direction sets

the number of steps, and that number grows with the **condition number** $\kappa = \lambda_{\max}/\lambda_{\min}$ of the Hessian. For real loss surfaces κ can be in the thousands.

There is a second problem. $\nabla \mathcal{L}$ is a sum over the whole training set, so every step requires a full pass over the data. Gradient descent fails both of our must-haves.

10. A2 — Momentum

A2 Momentum

Exponential moving average (EMA) of the gradient $g_t = \nabla_{\theta} \mathcal{L}(\theta_t)$:

$$m_t = \beta m_{t-1} + (1 - \beta) g_t, \quad \theta_t = \theta_{t-1} - \eta m_t.$$

The moving average covers roughly the last $1/(1 - \beta)$ gradients, about ten steps at the usual $\beta = 0.9$. Consider the zig-zag again. The component across the valley changes sign at every step, so it cancels in the average, while the component along the valley keeps the same sign and adds up. Momentum therefore damps the oscillation caused by poor conditioning and speeds up progress along the valley, at the cost of storing one extra copy of θ .

11. A3 — Stochastic gradient descent

We can also attack the cost directly, by not computing the exact gradient at all. Instead we estimate it on a random minibatch of B examples.

A3 Stochastic gradient descent

$$\text{error on the gradient} \propto \frac{1}{\sqrt{B}}, \quad \text{cost per step} \propto B.$$

These two scalings are what makes the method work. Halving the error in the gradient costs four times as much compute, so an accurate gradient is expensive. It also turns out to be unnecessary: for a fixed compute budget, many cheap noisy steps make more progress than one exact step. All large models are trained this way.

Where the anisotropy comes from

Poor conditioning can also come from the data rather than from the loss. Take a linear model in two features with a squared error:

$$\hat{y} = \theta_1 x_1 + \theta_2 x_2, \quad \mathcal{L} = \frac{1}{2} (\hat{y} - y)^2, \quad e \equiv \hat{y} - y,$$

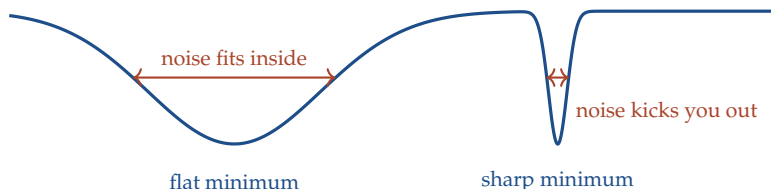
$$\frac{\partial \mathcal{L}}{\partial \theta_1} = e \cdot x_1, \quad \frac{\partial \mathcal{L}}{\partial \theta_2} = e \cdot x_2.$$

The gradient component for θ_i is proportional to the feature x_i . So if the features are physical quantities of very different size, as in the example on the board with q of order unity, a mass $M \sim 10^{12}$ and a power spectrum $\text{PS} \sim 10^4$, the gradient components differ by twelve orders of magnitude. A single η cannot work for all of them: it either diverges along the large component or makes no progress along the small one.

The ravine of Section A1 is not a special case. It appears whenever the inputs or the parameters live at different scales, which for physical data is almost always. There are two remedies: standardise the features, which you should do in any case, or give the optimiser a per-dimension step size, which is what A4–A6 do.

An aside on the noise: sharp and flat minima

The noise in the gradient is not only a cost. Consider two minima of the same depth but different width:



If a basin is narrower than a typical stochastic step, the iterate is knocked out of it, while a broad basin is not. Stochastic gradient descent is therefore biased towards flat minima.

This is the bias we want. A flat minimum means the loss is insensitive to perturbations of θ , and to first order a shift from the training distribution to the test distribution acts like such a perturbation. A sharp minimum can fit the training set perfectly and fail just outside it. This is the generalisation question, the second gap left open by the universal approximation theorem, and we return to it next lecture.

12. A4 — AdaGrad

If a single global learning rate cannot serve dimensions of different scale, we can give each dimension its own, derived from the history of its gradients.

A4 AdaGrad

$$\theta_{t+1,d} = \theta_{t,d} - \left(\underbrace{\eta_t \frac{1}{\sqrt{s_{t,d}} + \varepsilon}}_{\text{effective learning rate}} \right) g_{t,d}, \quad s_{t,d} = \sum_{i=1}^t g_{i,d}^2.$$

$s_{t,d}$ accumulates the squared gradients in dimension d , and ε is a small constant that prevents division by zero.

A dimension whose gradients have been consistently large gets a small effective step, and one whose gradients have been small gets a large one. This is the correction the feature-scale example above called for, applied per coordinate, and it costs one extra copy of θ .

The problem with AdaGrad. $s_{t,d}$ is a running sum of non-negative terms, so it only grows, and the effective learning rate decreases monotonically towards zero whether or not that is appropriate. For convex problems this is desirable and can be proved to converge. For deep networks it means training stops making progress long before the model has finished

learning.

13. A5 — RMSProp

The fix is to replace the sum with an exponential moving average, so that s tracks a window of recent gradients instead of the whole history.

A5 RMSProp

$$s_{t+1,d} = \beta s_{t,d} + (1 - \beta) g_{t,d}^2$$

with the same update rule as AdaGrad.

Now s can decrease as well as increase, so the effective learning rate recovers when a dimension becomes flat. The optimiser adapts to the current region of the surface rather than to the whole path taken so far.

14. A6 — Adam

Adam combines the two ideas: momentum in the numerator, RMSProp in the denominator.

A6 Adam

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t, & s_t &= \beta_2 s_{t-1} + (1 - \beta_2) g_t^2, \\ \theta_{t,d} &= \theta_{t-1,d} - \eta_t \frac{m_{t,d}}{\sqrt{s_{t,d} + \varepsilon}}. \end{aligned}$$

Bias correction

Both moving averages start at zero,

$$m_0 = s_0 = 0,$$

so the early estimates are biased towards zero. After t steps with a constant gradient, $\mathbb{E}[m_t] = (1 - \beta_1^t) \mathbb{E}[g]$. We divide the bias out:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{s}_t = \frac{s_t}{1 - \beta_2^t},$$

and use these in the update. The correction only matters for $t \lesssim 1/(1 - \beta)$, but with the standard $\beta_2 = 0.999$ that covers the first thousand steps, which is when a badly scaled step is most likely to spoil the run. The usual defaults are $\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\varepsilon = 10^{-8}$.

Measured against the wish list, Adam converges fast, stores two extra copies of θ and works at its default settings on problems it has not been tuned for. It gives up the convergence guarantees available for the convex methods. This is why it is the standard choice.

15. A7 — Muon, and what an update does to the function

A1–A6 all treat θ as a flat list of D numbers. But the parameters come in matrices, and a matrix acts on vectors. It is therefore worth asking what a step does to the function, rather than to the parameters.

Take the simplest example, $y = Wx$ with $x \in \mathbb{R}^2$ and $W \in \mathbb{R}^{2 \times 2}$, and write the gradient as a matrix,

$$G_{ij} = \frac{\partial \mathcal{L}}{\partial W_{ij}}, \quad W_t = W_{t-1} - \eta G.$$

The change in the output is then

$$\Delta y = (W - \eta G)x - Wx = -\eta \underline{G}x.$$

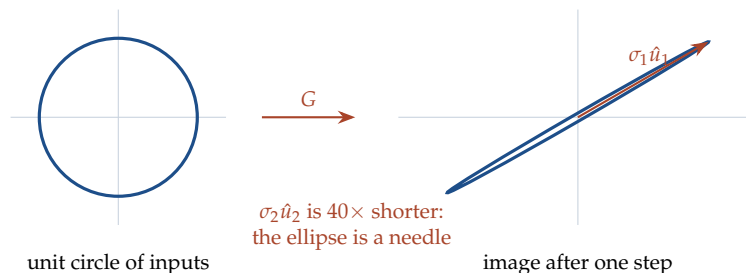
The effect of the step on the function is G acting on x . This is a linear map, and its gain depends on the direction of x . For an ordinary-looking G :

$$G = \begin{pmatrix} 1 & 1 \\ 1 & 1.1 \end{pmatrix},$$

$$x = \begin{pmatrix} 1 \\ 1 \end{pmatrix} \Rightarrow Gx = \begin{pmatrix} 2 \\ 2.1 \end{pmatrix} \quad (\times 2), \quad x = \begin{pmatrix} 1 \\ -1 \end{pmatrix} \Rightarrow Gx = \begin{pmatrix} 0 \\ -0.1 \end{pmatrix} \quad (\times 0.1).$$

One step amplifies one direction by about a factor of 2 and shrinks another by about a factor of 0.1. The singular value decomposition makes this explicit:

$$G = \sigma_1 (\hat{u}_1 \hat{v}_1^\top) + \sigma_2 (\hat{u}_2 \hat{v}_2^\top), \quad \sigma_1 \approx 2.05, \quad \sigma_2 \approx 0.05.$$



A single η has to be small enough to keep σ_1 under control, which means almost nothing happens along $\hat{v}_2$. This is the condition-number problem of Section A1 again, but now inside a single weight matrix. A per-coordinate method such as Adam cannot detect it, because it sees D separate numbers rather than the matrix they form.

The idea behind Muon. If the update acts with very different strength in different directions, we can flatten its spectrum before applying it so that every direction receives a comparable step. We stopped at this point in the lecture. How the flattening is done in practice, and whether it holds up at scale, is covered in this week's reading (Jordan et al.; Liu et al.).

Where this leaves us

The universal approximation theorem tells us a good network exists, and leaves two questions open. We spent today on the first. One idea runs through A1–A7: a single learning rate η is being asked to do a job that varies by direction, whether across parameters (A4–A6), across time (A2, A5) or within a single weight matrix (A7). Much of the progress in optimisation comes from finding another place where that assumption fails.

Next lecture we look at the gradient itself, $\nabla_{\theta}\mathcal{L}$, and then at generalisation: dropout, weight decay, grokking and double descent, and what they tell us about optimisation.

This week's reading. Jordan et al., *Muon* • Liu et al., *Muon is scalable* (arXiv:2502.16982).